

SIMATS ENGINEERING ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:50

Annexure A

Scenario Based Assignment

Code of Conduct:

I Shanmuganathan 192511347 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

20+20+23+18+19

90



① Scenario : Adversarial Search

(i) How minimax algorithm model this problem:-

- * Game Tree representation the game is modeled as a tree where nodes represent board states and edges represent valid moves.

- * Two player alternation player alternate turns max (the AI) tries to maximize the score, while Min (the human opponent) tries to minimize it.

- * Terminal states the leaf node represent game outcomes (win, loss or draw) assigned a precise numerical value.

- * Backup values minimax propagates utility values upward from the leaves to the root by assuming both player play perfectly at every step.

(ii) Computing challenge of Large Game Trees

- * Exponential growth the size of the game tree grows exponentially based on the branching factor b (average moves per turn) and depth d (turns ahead)

* Time Complexity:- $O(b^d)$ operation make Searching to terminal leaves impossible within strict time limits on low power table.

* Space Complexity:- $O(bd)$ for depth First Search (DFS) can still exhaust table memory, if no structural optimization or depth limits are missing.

* Hardware Constraints: Table have weaker CPU's and thermal limit, slowing down heavy recursive tree transversal.

(ii) How Alpha - Beta pruning Improves:-

* Eliminating Subtree:- It skips evaluating branches that cannot influence the final decision, reducing the effective branching factor.

* Alpha α : The minimum score that the max player is already assured of securing

* Beta β : The maximum score that the min player is already assured of holding.

* Pruning Condition:- A branch is instantly cut off

Evaluate Function Design:-

An effective evaluation function $f(s)$ estimates the utility of non-terminal states.

$$f(s) = w_1 \cdot \text{piece count} + w_2 \cdot \text{mobility} + w_3 \cdot \text{positional adv}$$

- * **piece count:** Different between AI piece and opponent piece to represent material dominance.
- * **mobility :** Number of legal move available to the AI minus those available to the opponent, preventing entrapment.
- * **positional Advantage:** Higher weight given to controlling critical zone (eg: Centre square or back row in chess).

(v) Recommend Strategy:-

- * **Iterative Deepening :** Run DFS progressively deeper ($d = 1, 2, 3, \dots$) if the strict time limit expires instantly return the best move from the last.
- * **Transposition Table :-** Use hash tables to store previously evaluated board positions, preventing redundant calculation of identical states reached through different move sequence.

2) Scenario: Heuristic Search & Path Finding

(i) Greedy Best First Search Behavior:-

* Mechanism: Expands nodes solely based on the heuristic estimate $h(n)$ (straight-line distance to the accident).

* Strength: Extreme fast execution with minimal computation overhead. Finding a path quickly by driving directly towards the target.

* Weaknesses under uncertainty:- High prone to getting stuck in local dead ends. (eg. a road blocked by sudden closure or a lake) because it completely ignores historical path cost $g(n)$. It is not guaranteed to find the shortest or safest

(ii) How it handles the situation:-

* Evaluating function:- Evaluates nodes using $g(n) + h(n)$

* Cost Components:- Accumulates the actual cost travelled so far factoring in dynamic variable cost like traffic congestion.

Heuristic:- Uses straight line distance to estimate the remaining travel cost.

(ii) Impact of Heuristic Accuracy:-

* Admissible heuristic:- An Admissible heuristic never overestimates the actual cost to the goal ($h(n) \leq h^*(n)$). Straight line distance is an admissible path. This guarantees A* will find the mathematically optimal route.

* Consistent: A heuristic is consistent if the estimate to the goal from node A is less than or equal to the step cost. This ensures that only A* expands a node. The optimal path to that node has been found.

(iv) Effect of a Dynamic Change of Search:-

In Validation:- Sudden road closures or spikes in ~~traffic~~ immediately invalidate the pre-calculated path.

Computational penalty:- Traditional A* must stop and recalculate the entire path from scratch, wasting critical seconds while the vehicle is moving.

Graph update cost:- Modifying edge weights dynamically



* Recommendation: Lifelong planning A (LPA)* or p* like

* Justification :- These incremental variants of A* retain the previous search tree, when a road closes they only recalculate the specific parts of the path affected by the local changes rather than replanning for research.

3) Scenario : Local Search & optimization

(i) Hill Climbing in this scenario and its limitations

* Mechanism :- Starts with an initial valid schedule and makes small incremental changes. It accepts the change only if it reduces cost.

* Limitation :- It behaves greedily and strictly moves uphill towards better states. If no adjacent swap improves the schedule, it stops immediately without looking ahead.

(ii) Local Search Anomalies

* Local maxima (minima) : A schedule configuration that is better than all immediate adjacent adjustments but far more expensive than the global ideal.
Eg: Swapping local crew members

but restructuring a multi-city flight sequence would
save millions.

plateaus: A flat region of the search space where many different crew swaps result in the exact same operational cost. Eg. Shifting a standby crew member between identical airport hotels offers zero net financial changes, leaving the algorithm drifting without direction.

Ridges: A sequence of local peaks where the optimum is narrow and sloped. Eg. optimizing a tight schedule sequence where navigating any directional except a high precise simultaneous multi-crew adjustment result in immediate regulatory violation.

(iii) How Advanced Approaches Overcome Limitation:

* **Simulated Annealing:** Introduces controlled randomness. ~~Early~~ in the search, the algorithm deliberately accepts worse scheduling options with a probability governed by a temperature parameter. As the temperature cools, it gradually focuses purely on optimizing, allowing it to jump out of local minima traps.

Genetic Algorithm:- Maintain a large number of Alternative Schedules Simultaneously. It combines part of high performing Schedules through Crossover and introduces random mutation to explore completely new scheduling Configuration.

Recommended Approach and Justification.

* Recommendation:- Genetic Algorithm GA Hybridized with local repair heuristics.

* Justification:-

* Scalability: GA Scales excellently across thousands of variable because it processes structural chunks of Schedules in parallel rather than single point to point modification.

* Adaptability: When a slight delay disrupts the network, a GA can seed its starting population with the pre-existing Schedule and rapidly evolve a repaired, complaint alternative within minutes avoiding global paralysis.